

Generating mediated queries

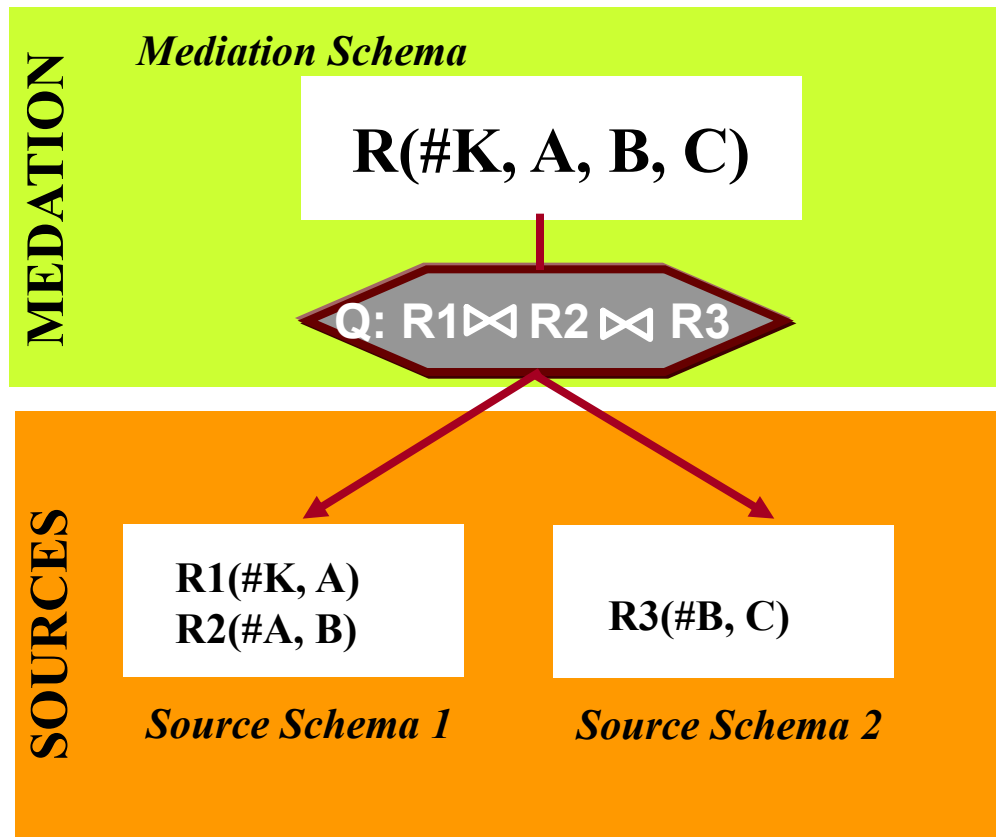
Vũ Tuyết Trinh
trinhvt@soict.hust.edu.vn

Hanoi University of Science and Technology

Data integration

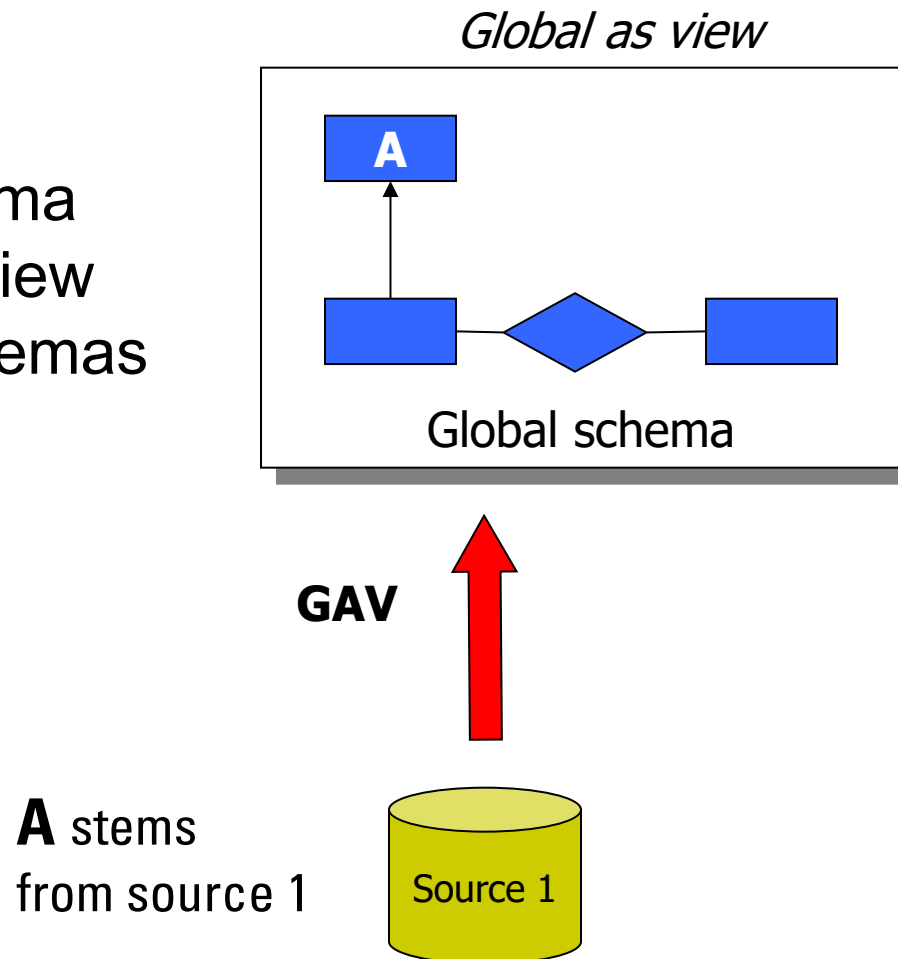
- Aims
 - An *integrated view* on multi data sources
 - A *transparent access* to multi data sources
- Architecture:
 - mediation ... P2P
- Problems
 - How to define the integrated view
 - How to locate data sources
 - How to forward *user queries*

Mediator-based architecture



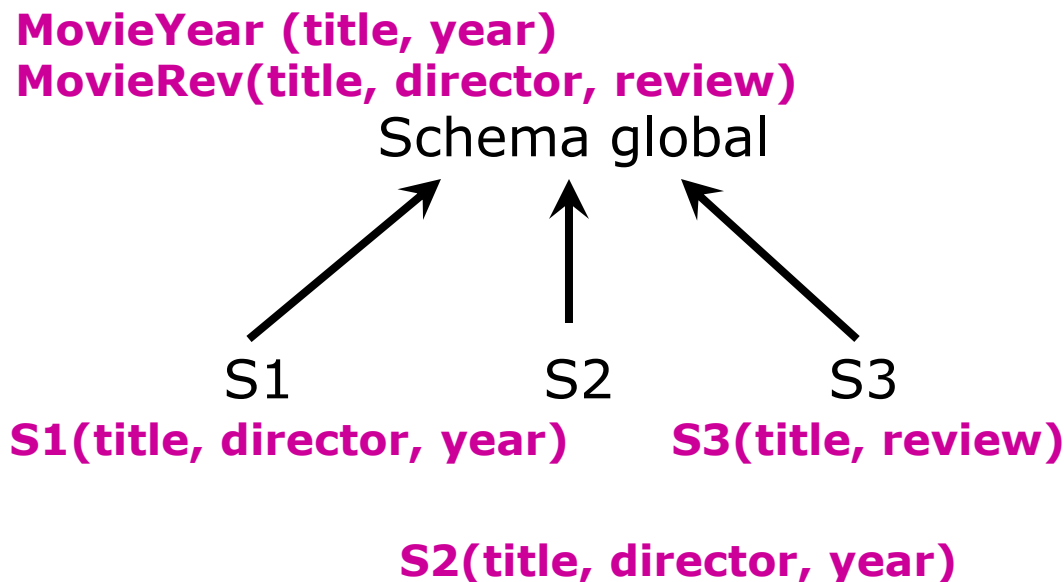
Integrating: Global As View approach

The global schema is defined as a view on exported schemas

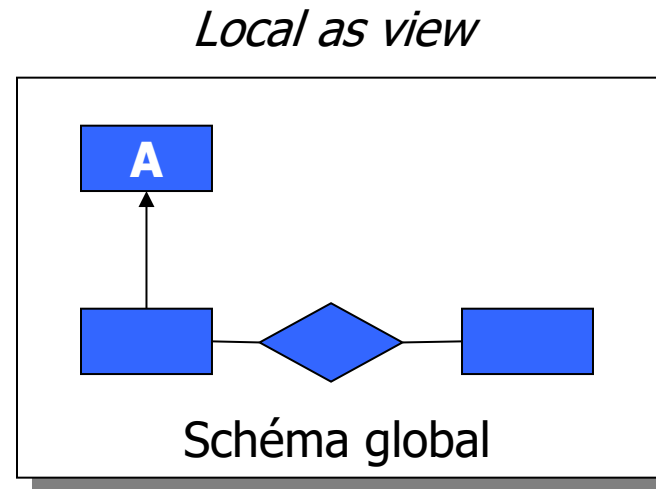


Example

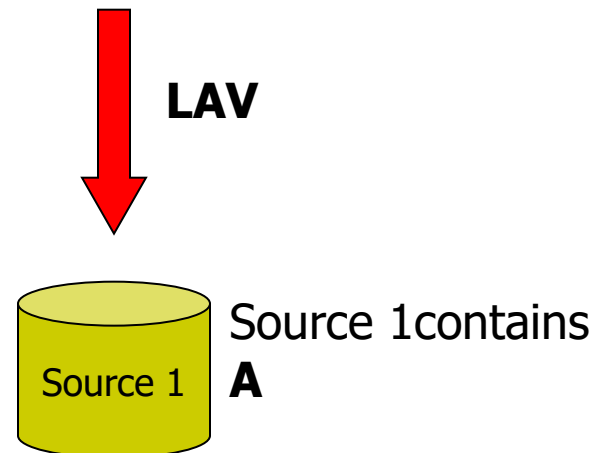
- **MovieYear** (title, year) :-S1(title, director, year)
MovieYear (title, year) :-S2(title, director, year)
- **MovieRev** (title, director, review) :-S1(title, director, year) & S3(title, review)
- **MovieRev** (title, director, review) :-S2(title, director, year) & S3(title, review)



Integrating: Local As View approach

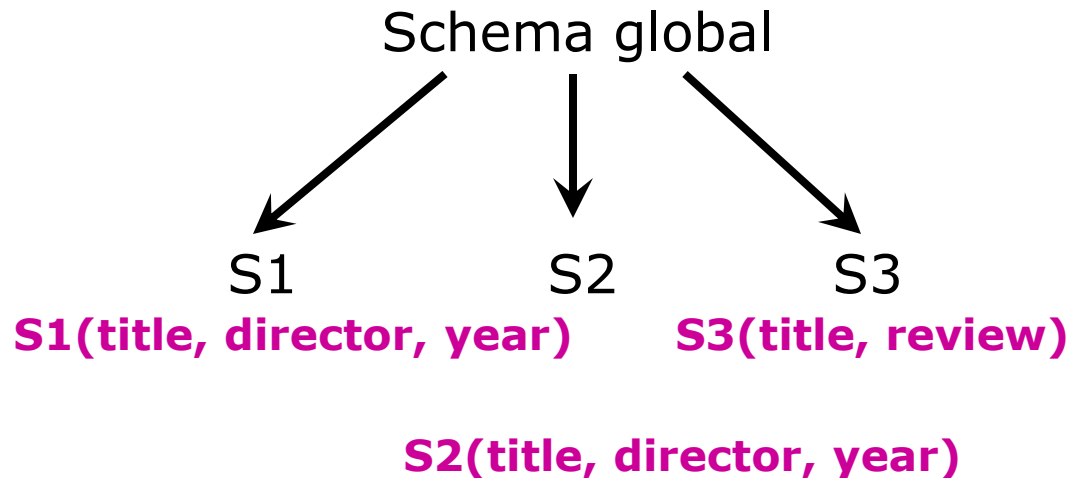


Exported schemas
are defined as views
on the global schema



Example

MovieYear (title, year)
MovieRev(title, director, review)



- **S1: V1**(title, year, director) :- MovieYear(title, year) & MovieRev(title, director, review)
- **S2: V2**(title, year, director) :- MovieYear(title, year) & MovieRev(title, director, review)
- **S3: V3**(title, review) :- MovieRev(title, director, review)

LaV - rewriting

- Query: look for reviews of films in 1997

q(title, review):- MovieYear(title, 1997) &
MovieReview(title, director, review)



q1(title, review):-V1(title, 1997,director) &
V3(title, review)

q2(title, review):-V2(title, 1997,director) &
V3(title, review)

...

Principle of rewriting in LaV

□ Inverse rules

V1(title, year, director) :- MovieYear(title, year) &
MovieRev(title, director, review)

→ MovieYear(title, year) :- V1(title, year, _)
MovieRev(title, director, ?) :- V1(title, _, director)

V3(title, review) :- MovieRev(title, _, review)

→ MovieRev(title, ?, review) :- V3(title, review)

...

□ Unfolding

q1(title, **review** - V1(title, 1997, _) & V1(title, _, director)

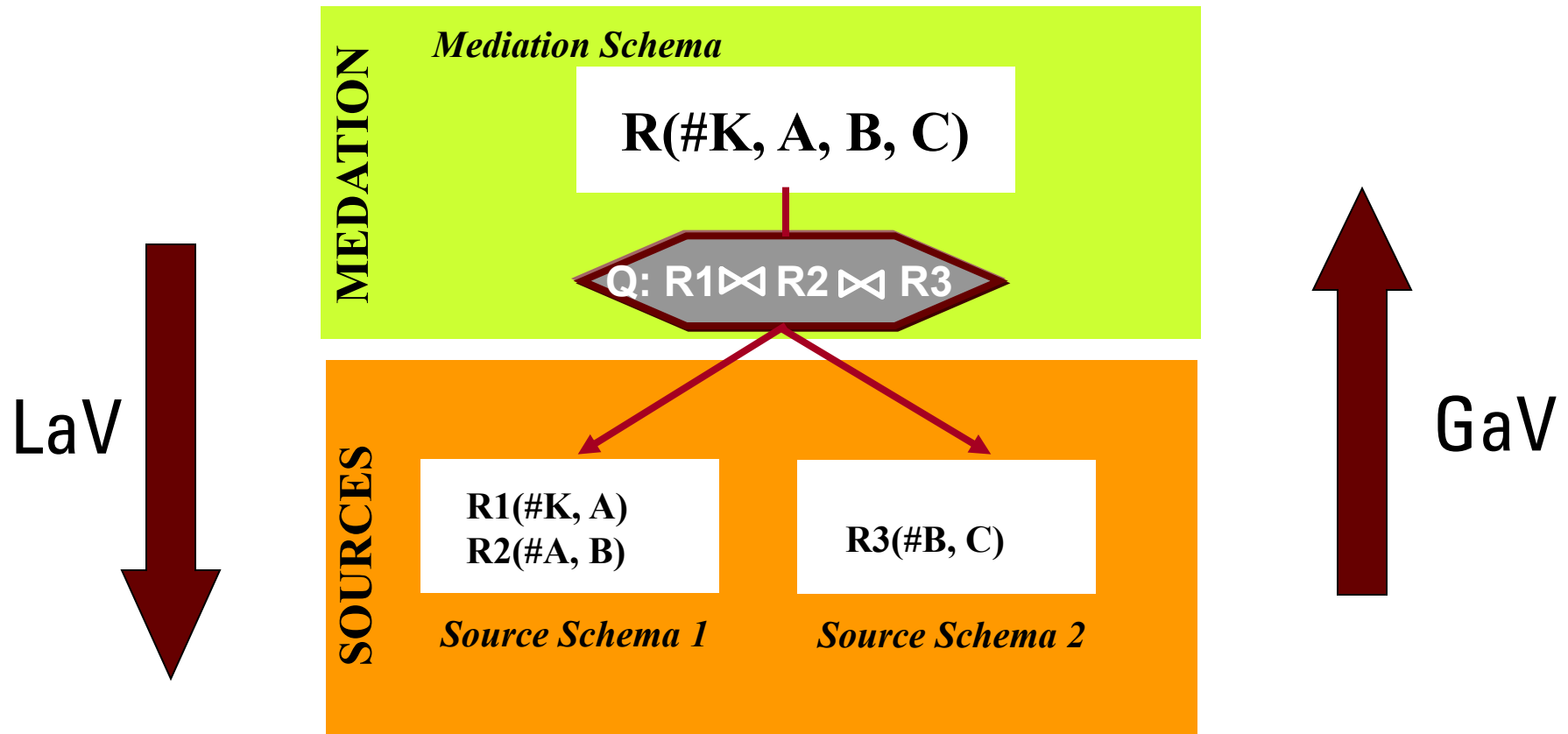
q2(title, review) :- V1(title, 1997, _) & V3(title, review)

...

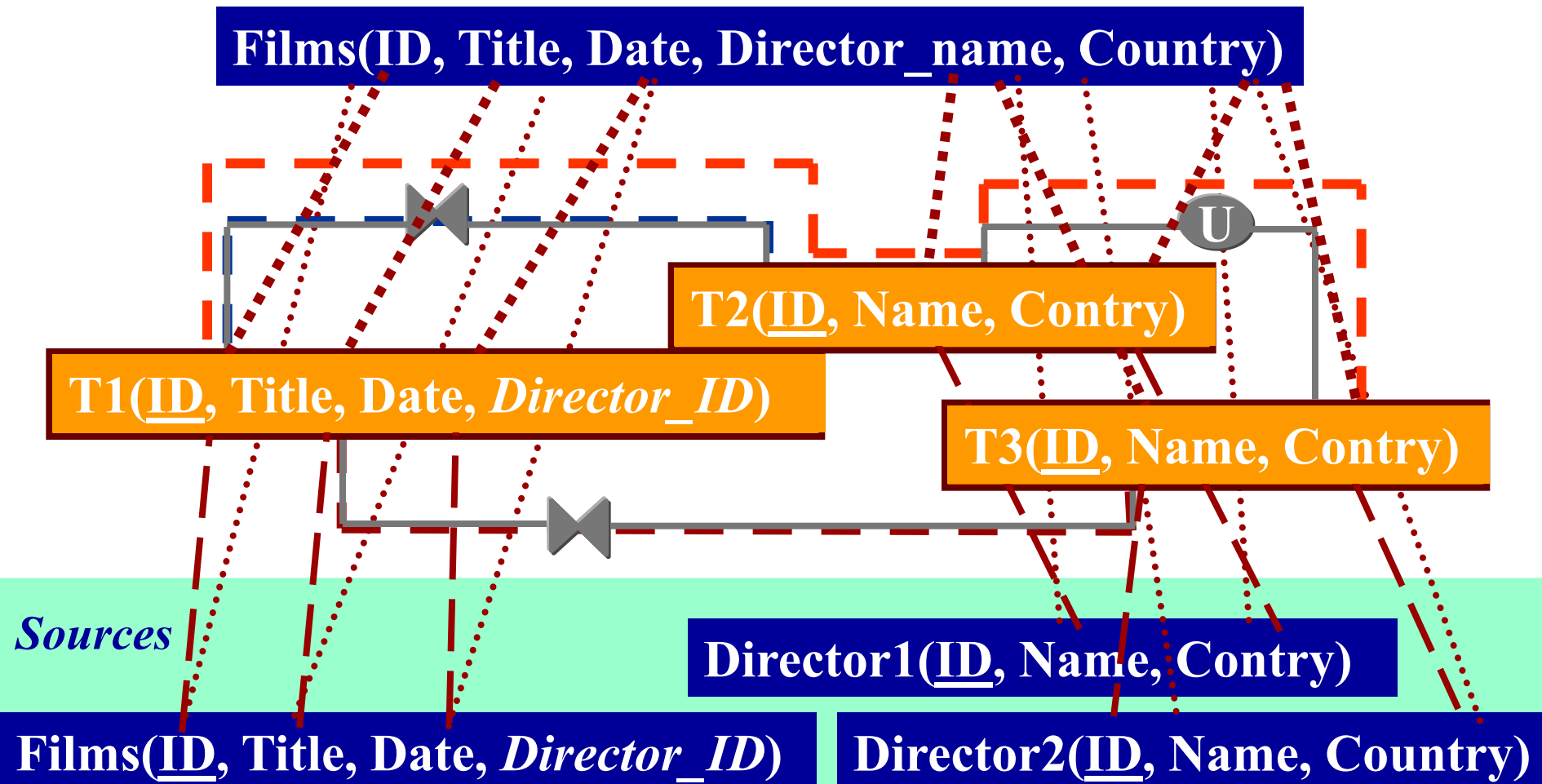
□ Cleaning

Check validity of unfolded queries

Comparaison: GaV vs. LaV

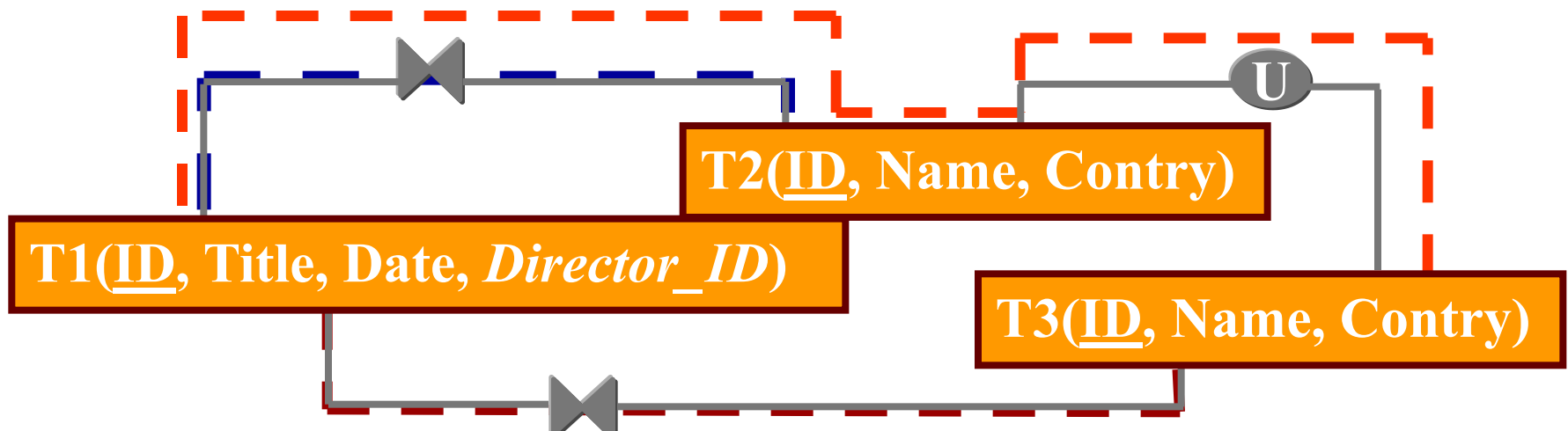


Generating mediation queries (toward an automatic approach)



Generating mediation queries (2)

Films(ID, Title, Date, Director_name, Country)



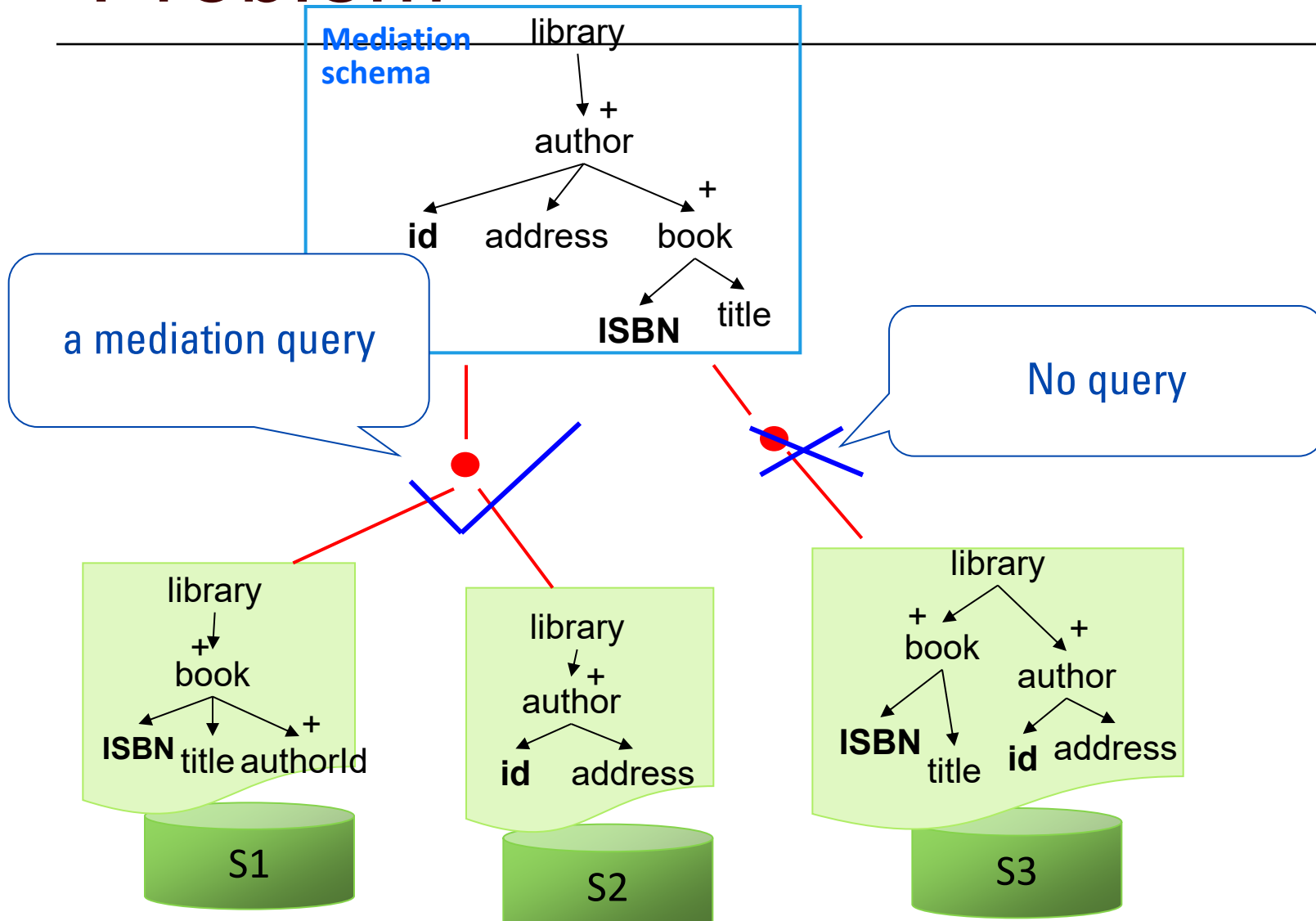
Mediation queries

$Q_1: \pi_{ID, Title, Date, Name, Country}(T1 \bowtie T2)$

Rewriting

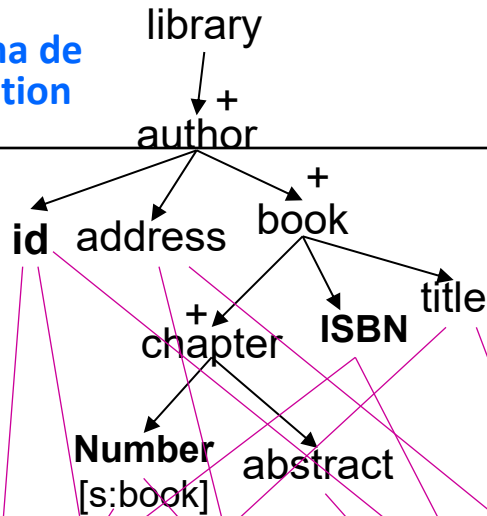
$Q_1: \pi_{ID, Title, Date, Name, Country}((\pi_{ID, Title, Date, Director_ID} \text{ Films}) \bowtie (\pi_{ID, Name, Country} \text{ Director1}))$

Problem



Assumption

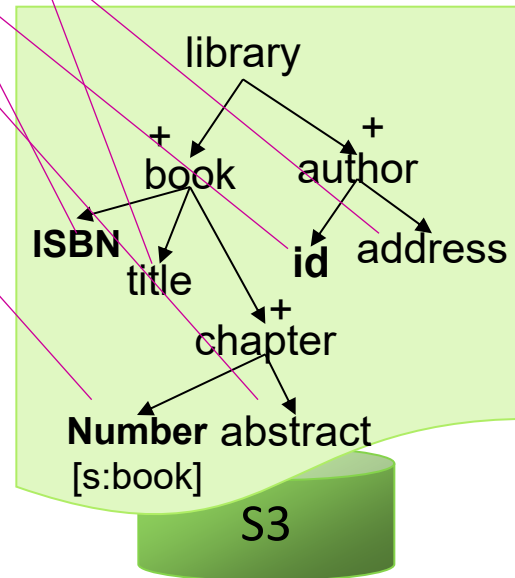
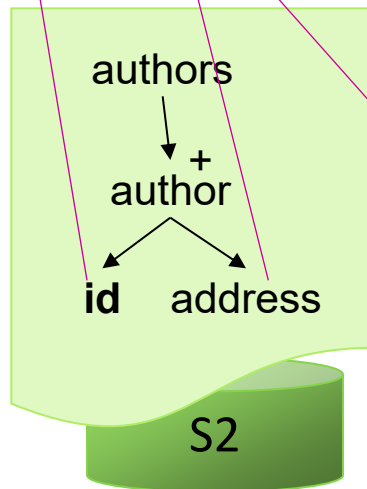
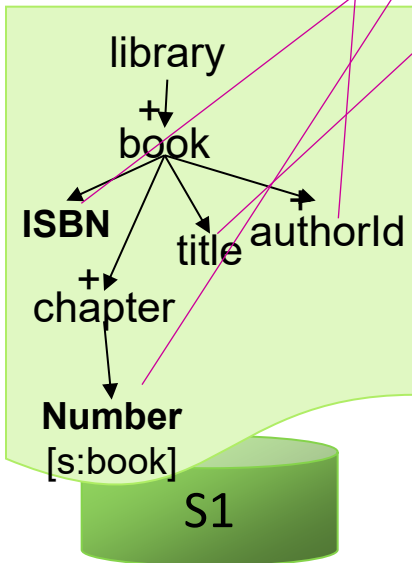
Schéma de médiation



Schema in XML Schema

Semantic correspondences

$SM/Library/Author/Id \cong S1/Library/book/AuthorId$

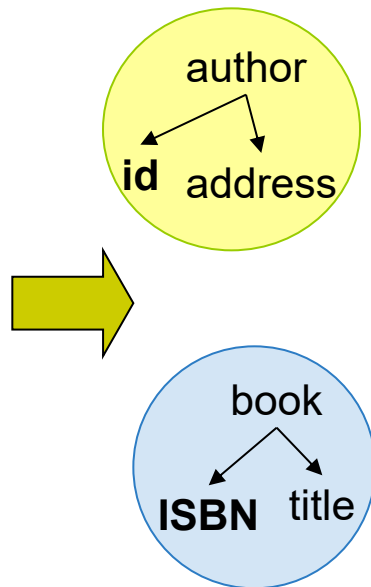
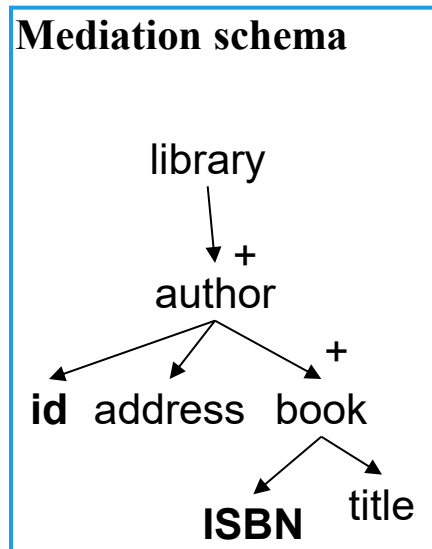


Generating mediation queries

□ Principle

- Decomposing the mediation schema into **sub-trees**
- Looking for partial mappings
- Identifying mediation queries : combining partial mappings

Mediation query

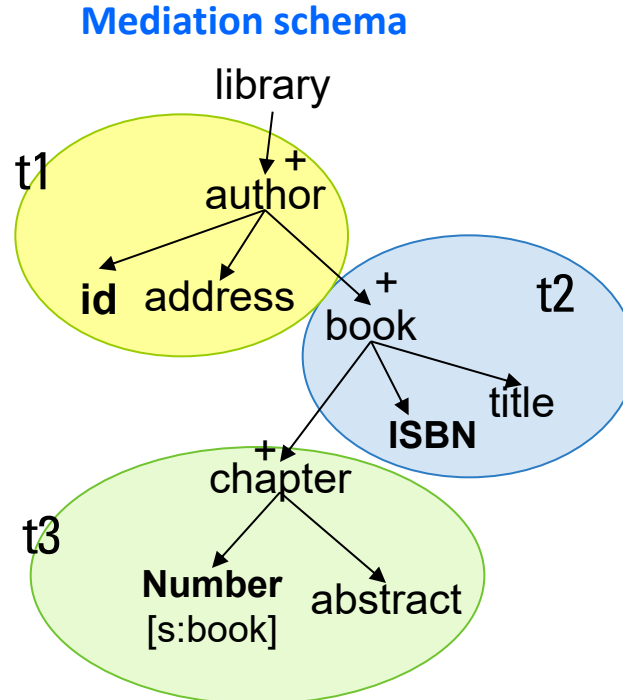


```

<Library>{
  for $id = distinct(S1/Library/book/authorId,
    S2/authors/author/Id)
  for $p1 in S1/Library/book/authorId
  for $p2 in S2/authors/author
  where $p1 = $p2/id and $p1 = id
  return <Author>{
    <id>{$p1/text()}</id>,
    <Address>{$p2/address/text()}</Address>
  }
  for $isbn = distinct(S1/Library/book/ISBN)
  for $p4 in S1/Library/book[authorId = $p1]
  where $p4/ISBN = $isbn
  return <Book>{
    <ISBN>{$p4/ISBN/text()}</ISBN>,
    <title>{$p4/title/text()}</title>
  }
}
  
```

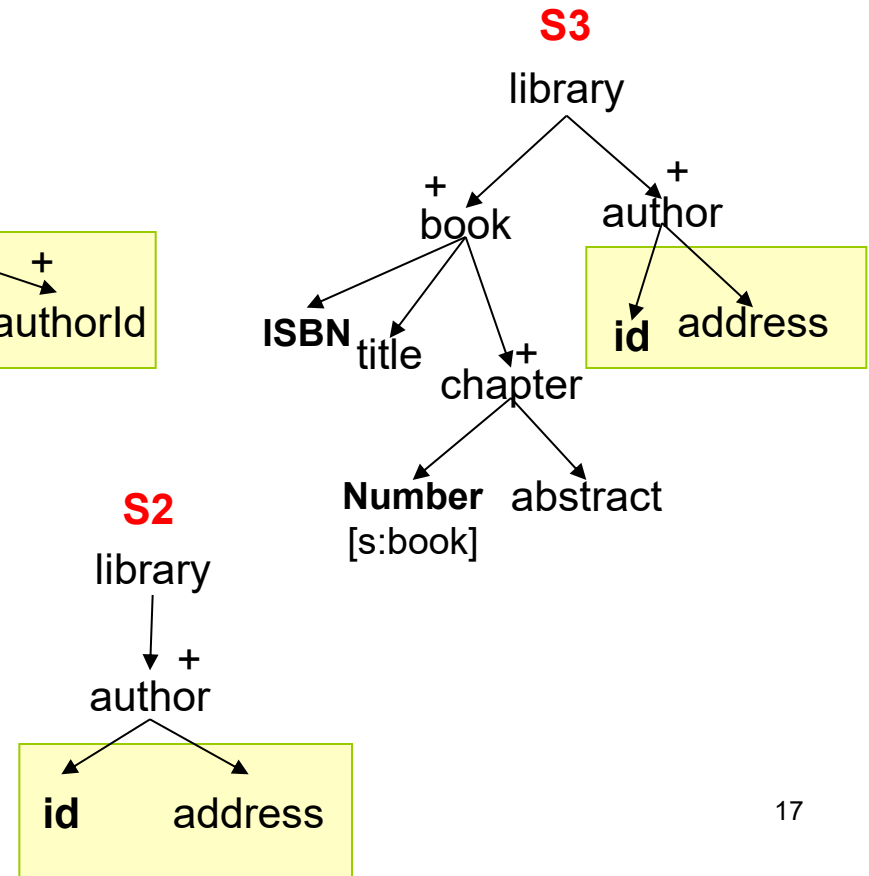
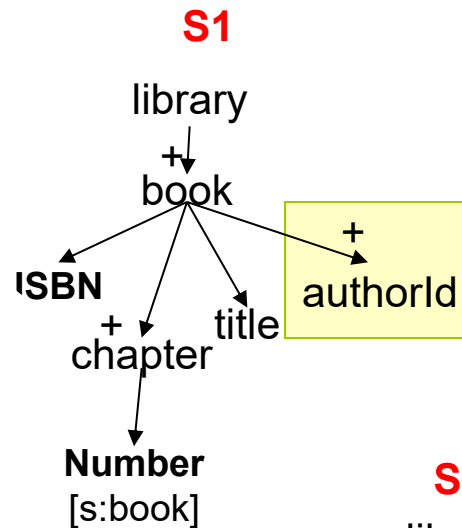
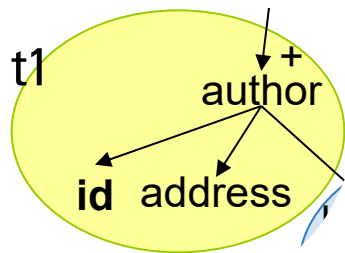
Decomposing

- **Decomposing the mediation schema into sub-trees having**
 - Root: a multi-value node or the root of mediation schema
 - Node: mono-value
 - Existing at least a text node in sub-tree



Partial mappings

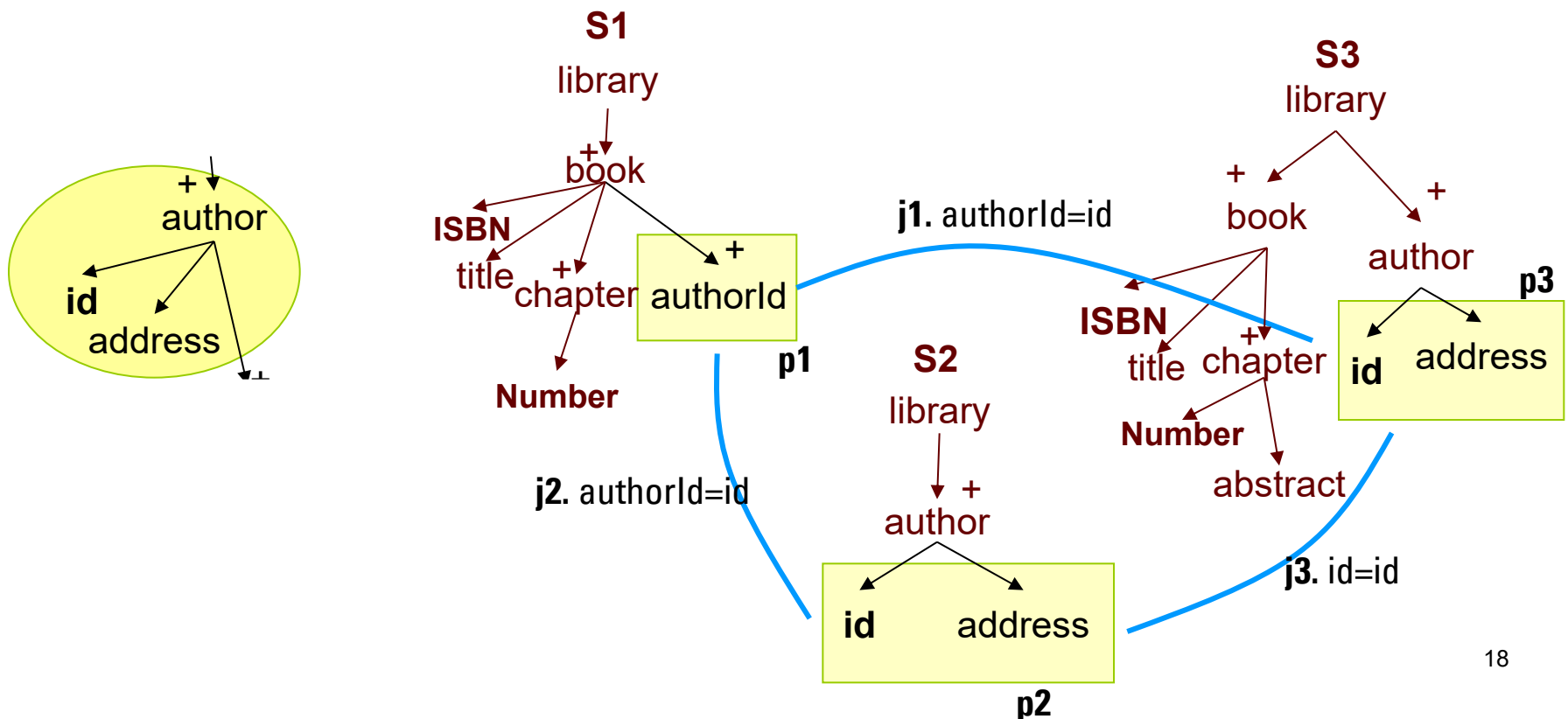
(1) For each sub-tree, identifying correspondent parties in sources



Partial mapping (2)

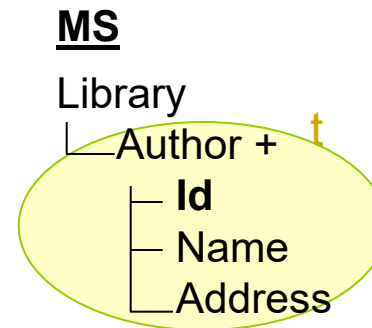
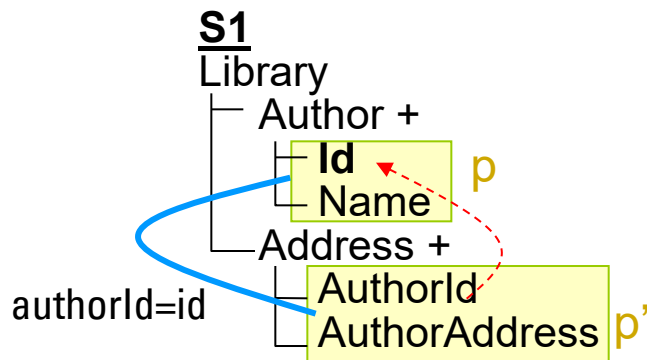
(2) Identifying possible join operations between sources

- semantic correspondences



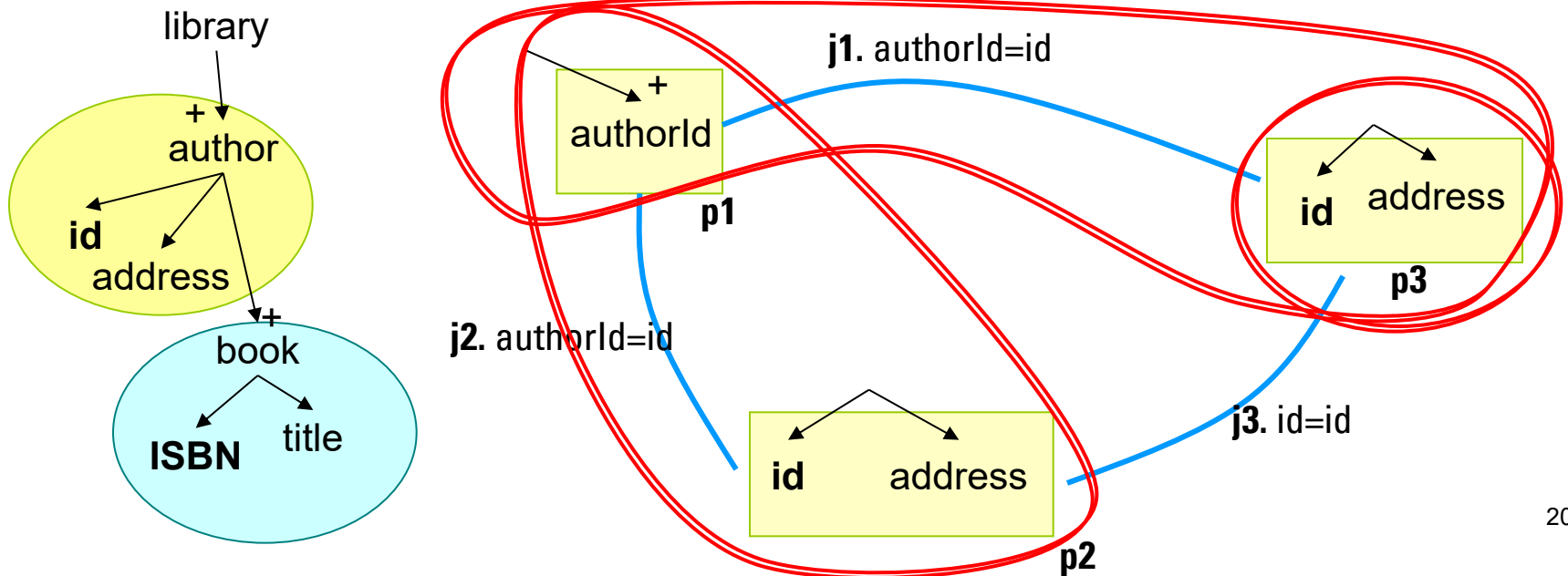
Partial mapping (3)

- Identifying possible inter-join operation
 - Key & reference key



Partial mapping (4)

- Graph of joins join :
 - Node ~ a source
 - axe ~ a join
- Partial mapping: set of sub-graph of joins



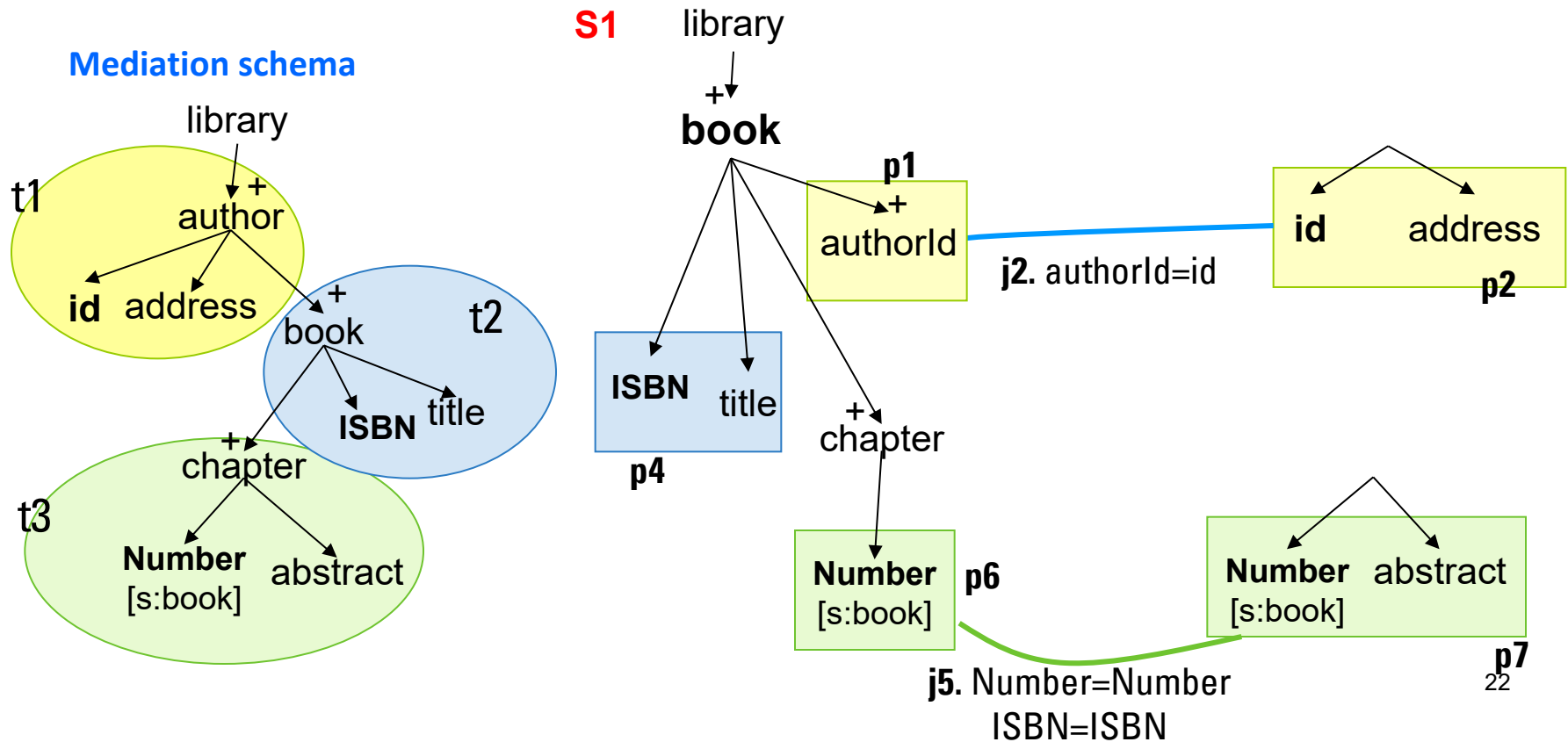


Generating mediation queries

- **A mediation query ~ a combine of partial mappings**
 - A partial mapping for each sub-tree
 - Having some mediation queries (with different semantics)
- **Union of 2 mediation queries ~ a mediation query**

Generating ... (2)

□ Verifying parent-child relations in sub-trees



Generating mediation queries

❑ Requête XQuery

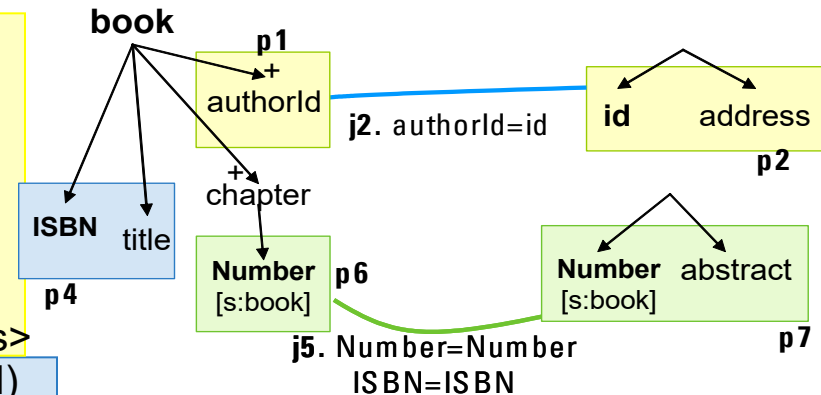
<Library>{

```
for $id = distinct(S1/Library/book/authorId,
                  S2/authors/author/Id)
for $p1 in S1/Library/book/authorId
for $p2 in S2/authors/author
where $p1 = $p2/id and $p1 = id
return <Author>{
  <id>{$p1/text()}</id>,
  <Address>{$p2/address/text()}</Address>
```

```
for $isbn = distinct(S1/Library/book/ISBN)
for $p4 in S1/Library/book[authorId = $p1]
where $p4/ISBN = $isbn
return <Book>{
  <ISBN>{$p4/ISBN/text()}</ISBN>,
  <title>{$p4/title/text()}</title>
```

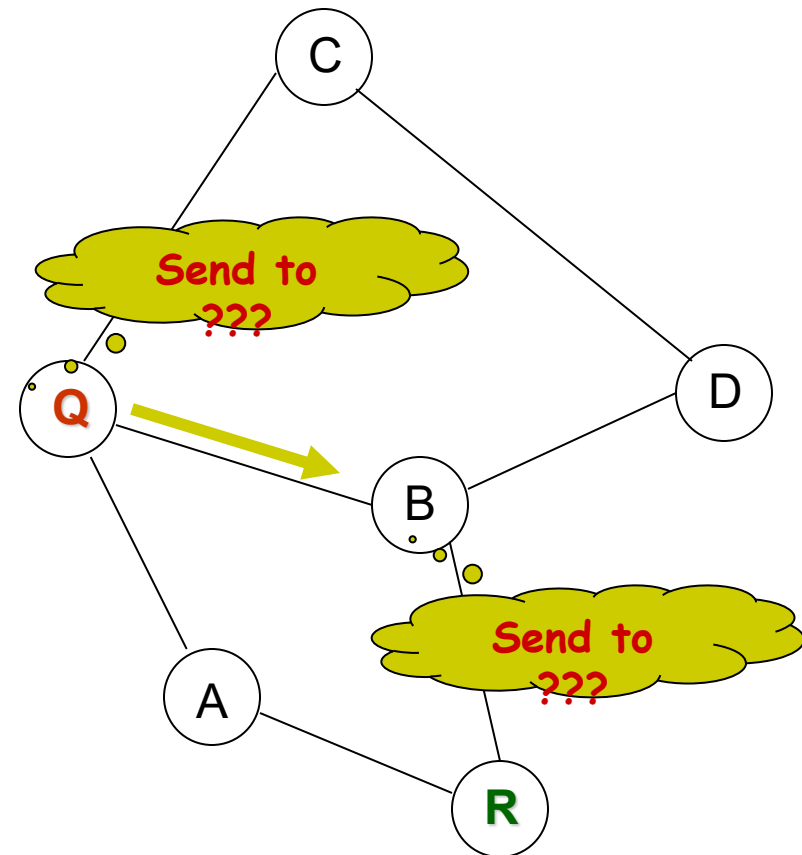
```
for $num = distinct(S1/Library/book/Chapter/Number,
                  S3/Library/book/Chapter/Number)
for $p6 in S1/Library/book[ISBN = $p4/ISBN]/Chapter
for $p7 in S3/Library/book/Chapter
where $p6/Number = $p7/Number and $p6/Number = $num
and $p6/PARENT::book/ISBN = $p7/PARENT::book/ISBN
return <Chapter>{
  .....
```

></Library>



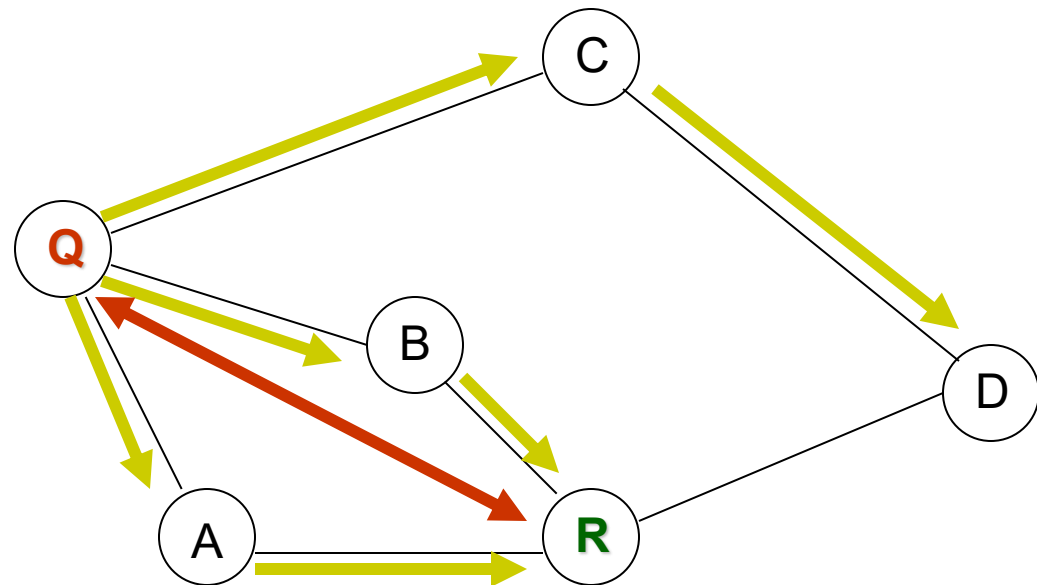
P2P-based architecture

- Integrating data peers?
- Choosing peer(s) to submit query?
- Choosing peer(s) to propagate query?



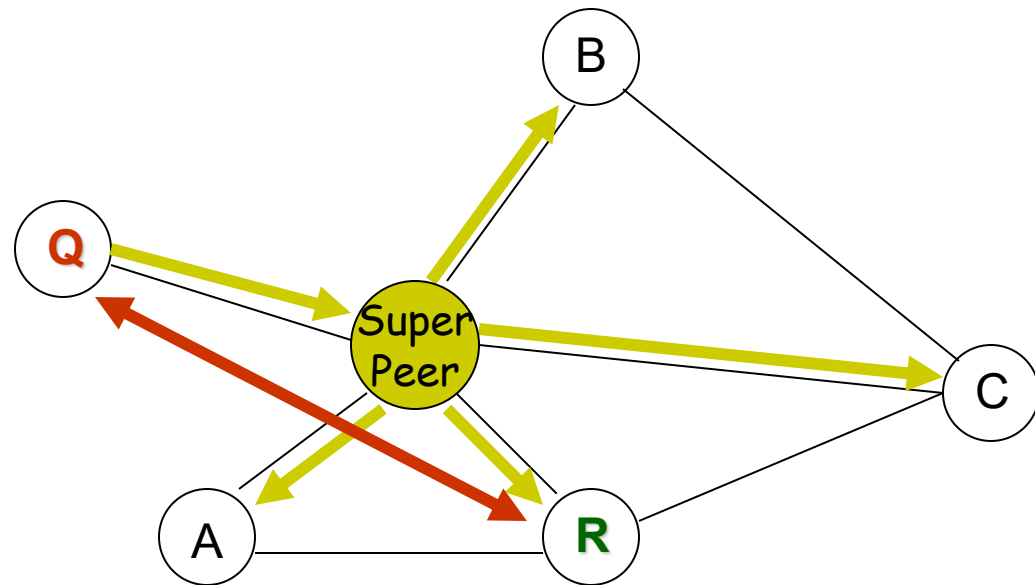
Purely decentralized

- No central co-ordination.
- propagating a query from a peer to neighboring peers



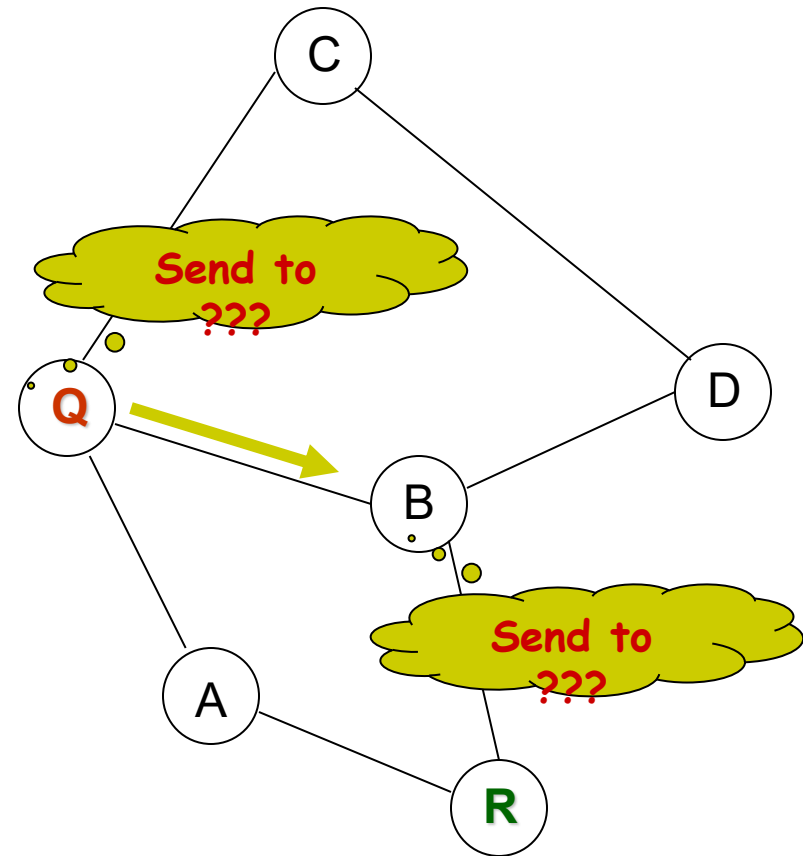
Partially centralized

- Some nodes assume role of local servers, called **SuperPeers**
- propagating a query from a super peer to its under peers



Query propagation in P2P

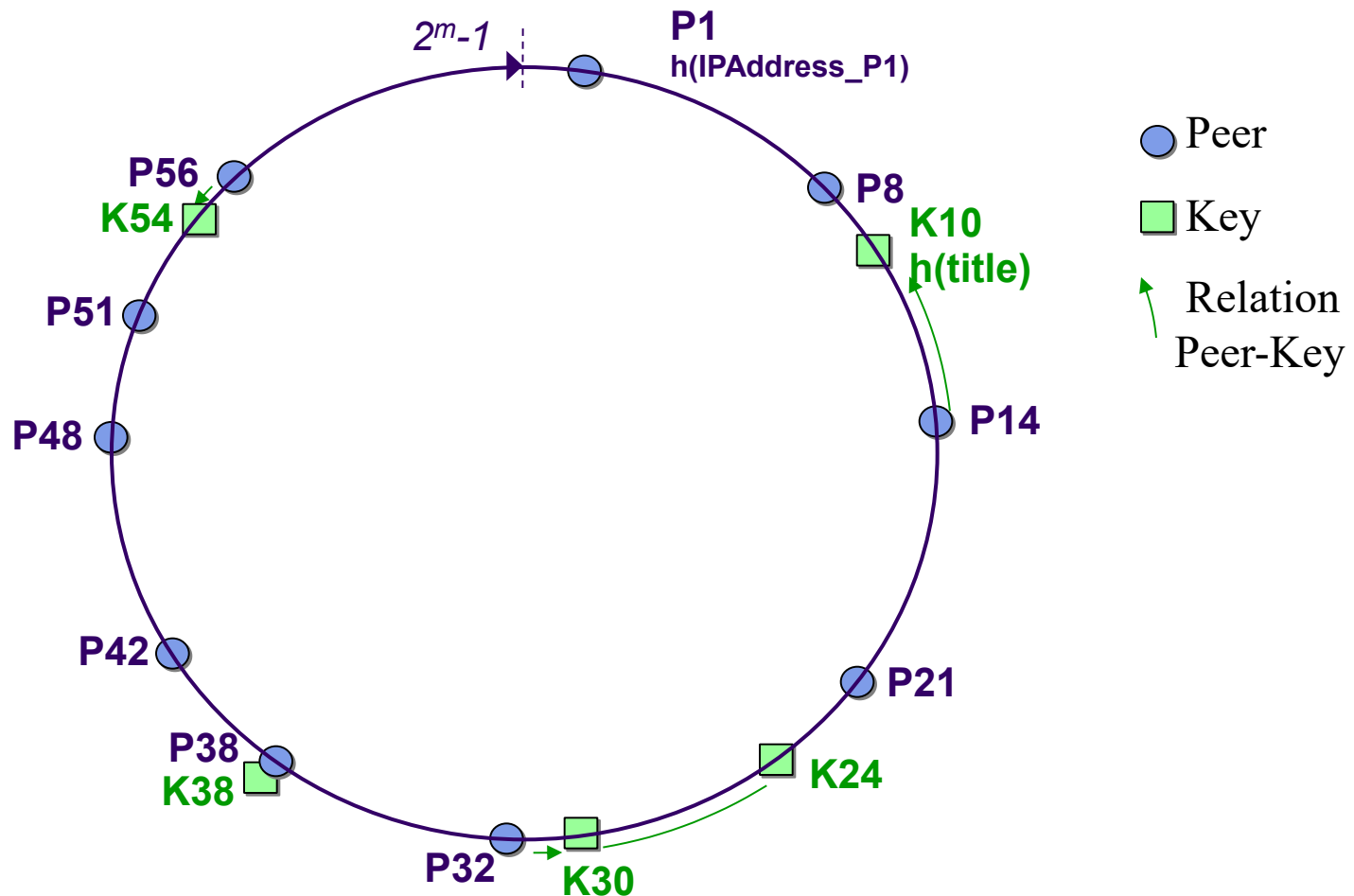
- Problem
 - Choosing node(s) to propagate query
- Solution
 - Structured
 - using index
 - Unstructured, “pure” decentralized
 - flooding technique, TTL
 - Partial centralized
 - Super-Peer as coordinator



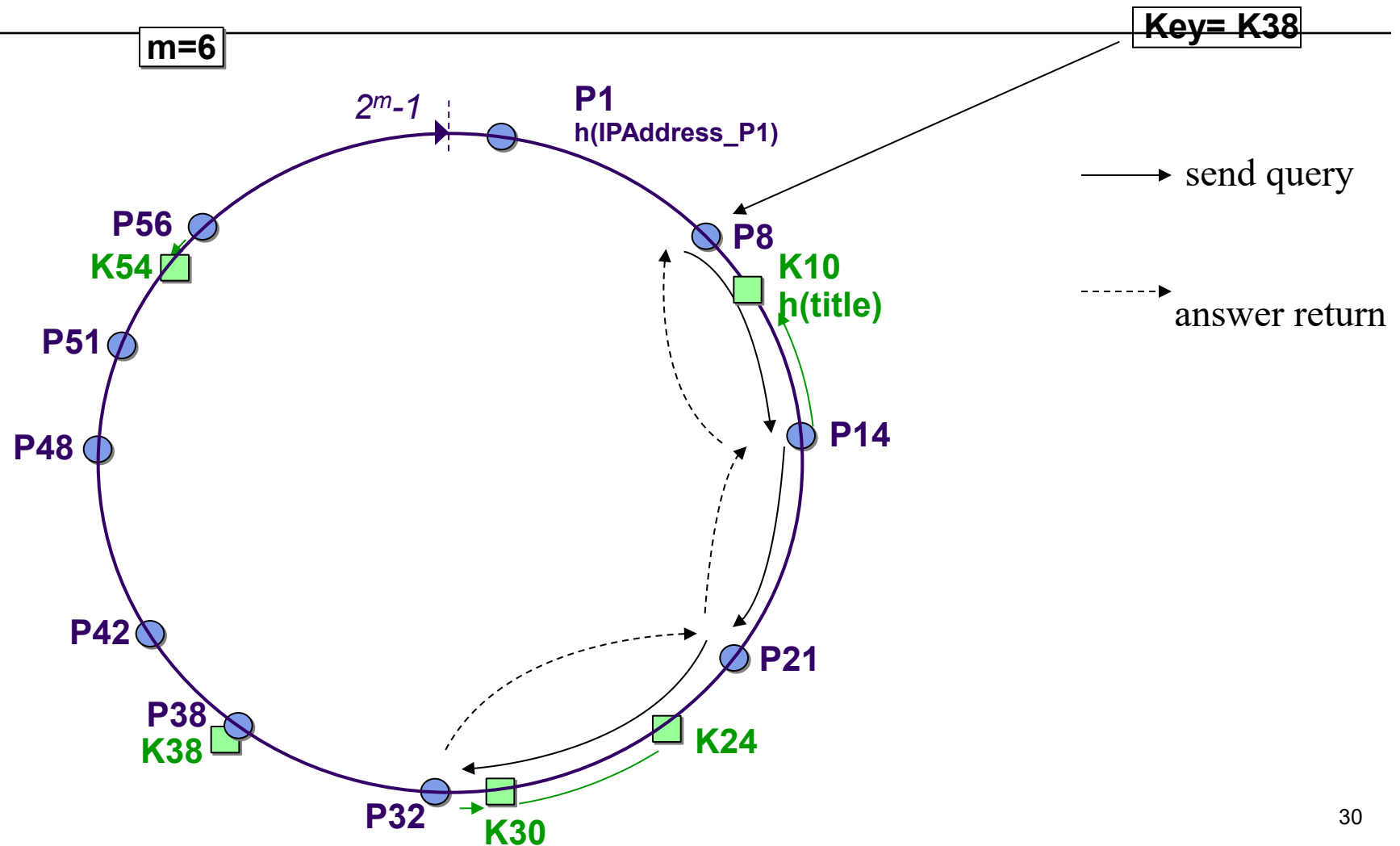
Structured systems

- (Distributed) Hash Table based Systems (DHTS)
 - Querying on key attributes
 - Using a hash function
 - Distributing hash table on peers
- Query propagation:
 - Using index
- Example: Pastry, Chord, Can, Tapestry

Chord: DHTs Example



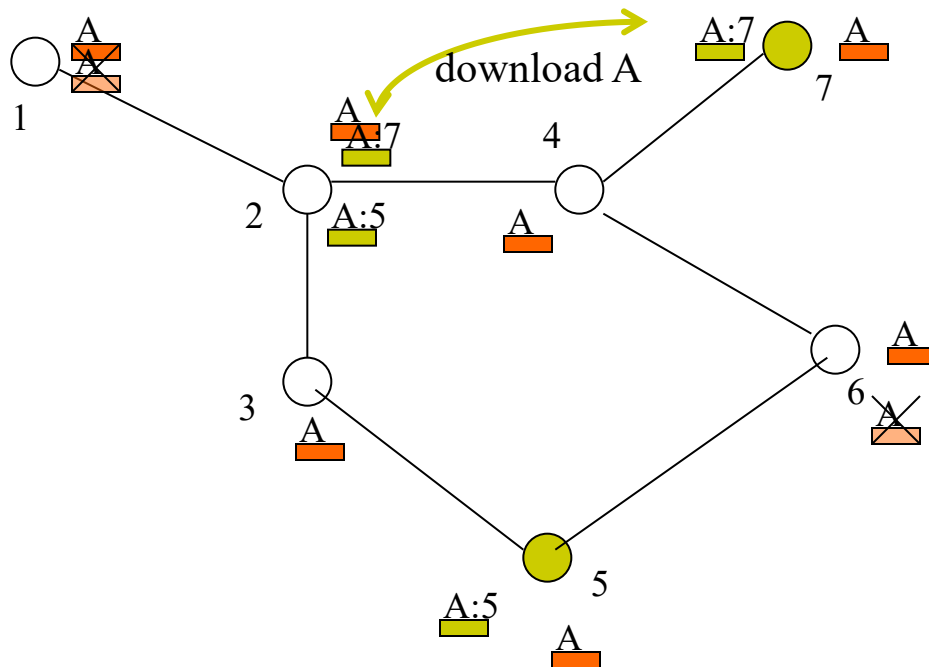
Queries in *Chord*



Unstructured systems

- Central or distributed **catalog** containing
 - Descriptive metadata (e.g., catalog records)
 - Administrative metadata (e.g. circulation records)
 - Structural metadata (e.g. serials record)
 - Rights metadata (e.g. shrink wrap license)
- Query propagation:
 - flooding technique
- Example: Napster, Gnutella

Gnutella search mechanism



Steps:

- Node 2 initiates search for file A
- Sends message to all neighbors
- Neighbors forward message
- Nodes that have file A initiate a reply message
- Query reply message is back-propagated
- File download

